

Gerenciamento de Estado

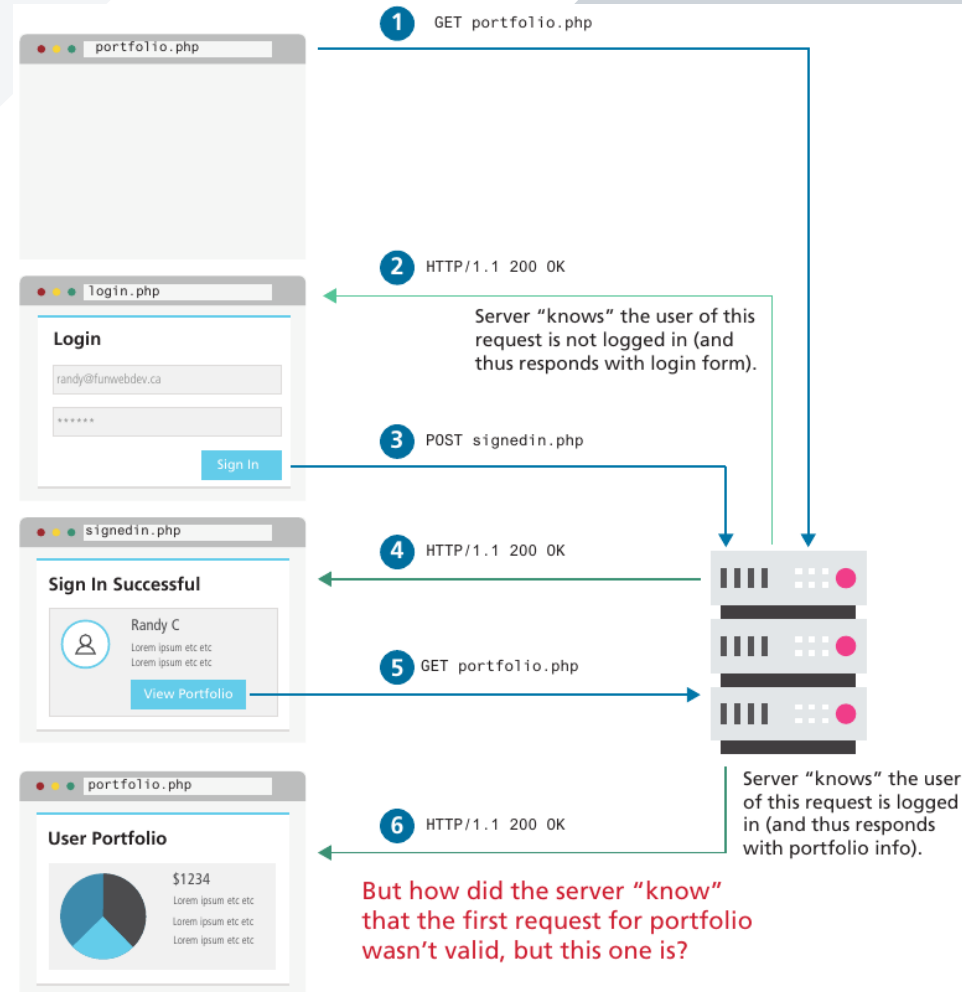
Programação para a Web 2

Aula 10

Evandro J.R. Silva

O problema

- Existem várias situações onde um site ou um serviço precisam de algum tipo de persistência de informação (a.k.a. “estado”), de forma que esta esteja disponível para outras páginas do site.
- Mas o HTTP é, intrinsecamente *stateless*. E agora?



A solução

- Existem algumas técnicas para lidar com as restrições impostas pelo HTTP:
 - Passando informação pelo próprio HTTP
 - URL
 - HTTP Header
 - Cookies
 - *Session State*
 - *Load Balancer*
 - Arquivos
 - *Caching*
- Nenhuma delas é perfeita, porém podem ser a mais adequada dependendo da situação.

Utilizando o HTTP

- URL
 - Através de *query strings*

```
GET /product/45653
```

```
GET /search/cassat/20
```

Node +
Express

```
app.get('/product/:id', (req,res) => {  
  const id = req.params.id;  
  ...  
})  
app.get('/search/:name/:limit', (req,res) => {  
  const name = req.params.name;  
  const limit = req.params.limit;  
  ...  
})
```

Utilizando o HTTP

- HTTP Header

- Alguns campos podem ser usados como o Content-Type

- Content-Type: application/x-www-form-urlencoded
 - Content-Type: application/json
 - Content-Type: audio/mpeg
 - Content-Type: image/png
 - Content-Type: multipart/form-data



Artist:

Year:

Nationality:

<form method="GET" action="process.php">

GET process.php?artist=Picasso&year=1906&nation=Spain http/1.1

Query string

<form method="POST" action="process.php">

```
POST process.php HTTP/1.1
Date: Sun, 15 Jan 2017 23:59:59 GMT
Host: www.mysite.com
User-Agent: Mozilla/4.0
Content-Length: 47
Content-Type: application/x-www-form-urlencoded
artist=Picasso&year=1906&nation=Spain
```

HTTP
header

Query string

Utilizando o HTTP

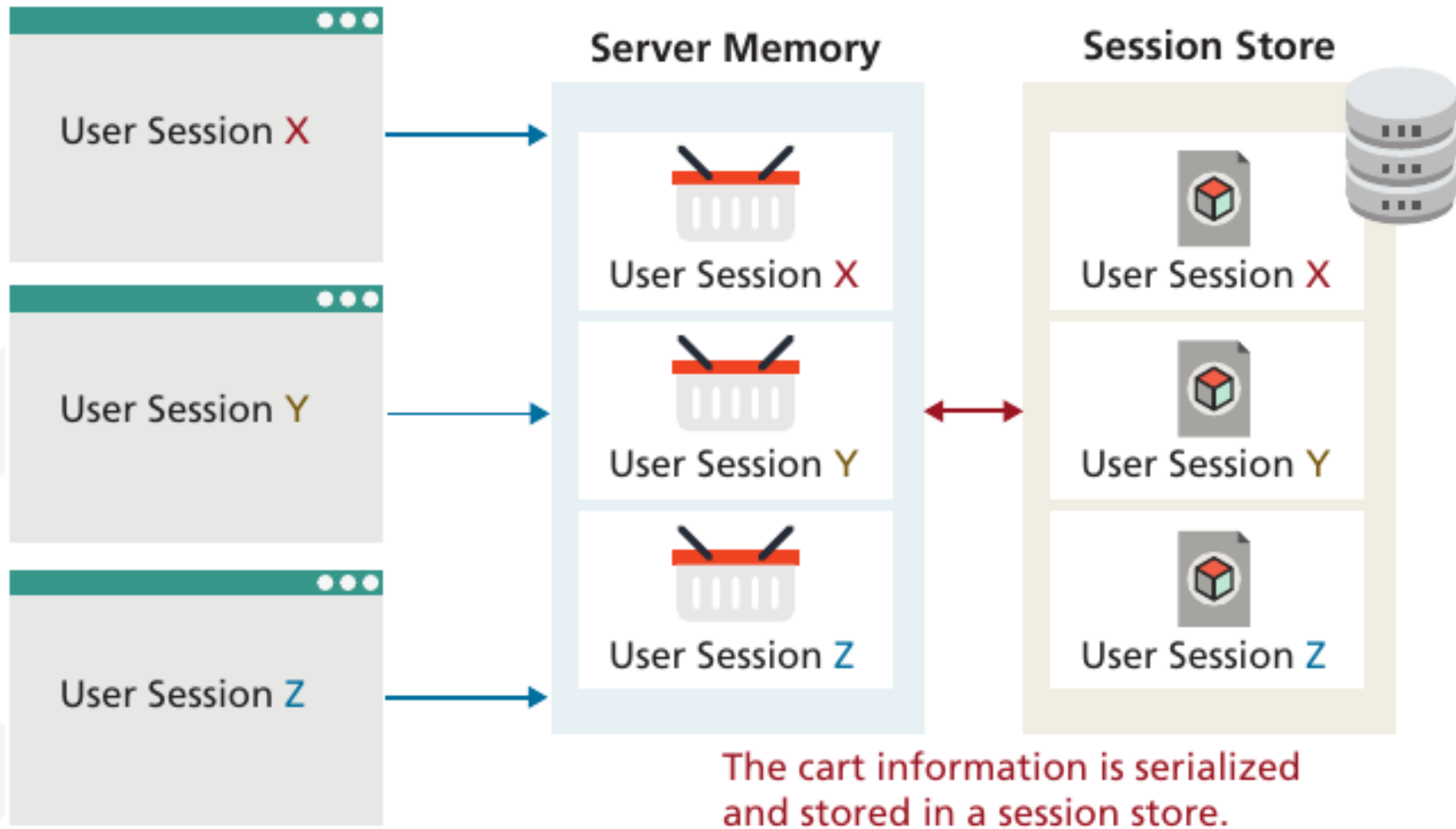
- *Cookies*

- Antes de enviar a resposta ao cliente, o servidor (sua API usando algum *framework*) serializa dados da sessão, criptografa e o inclui no cabeçalho de resposta do HTTP.
 - Enquanto o armazenamento de dados for pequena, essa solução funciona bem. Mas quanto mais informações são acrescentadas, pior fica do ponto de vista de desempenho (*cookies* são enviados em todas as trocas de mensagens HTTP).
- A limitação mais importante
 - Os usuários podem configurar os navegadores para recusar *cookies*.
 - Partes críticas do sistema não podem depender do uso de *cookies*.

Session State

- É um mecanismo de estado baseado em servidor para as aplicações web armazenarem e recuperarem objetos de qualquer tipo para cada sessão de usuário específica.
 - A aplicação web pega o identificador de sessão de uma requisição e, então, carrega os dados da sessão a partir de algum local de armazenamento (*data store*). Ao fim do ciclo de vida da requisição, pouco antes de enviar a resposta, a aplicação serializa os dados da sessão e os salva de volta no local de armazenamento.
 - O servidor web em si não mantém nenhum dado de sessão entre as requisições, o que faz que com seja uma alternativa *stateless* no contexto doo HTTP.
 - Alguns *data stores* usados para isso: Memcached, Redis, DynamoDB, Cassandra.

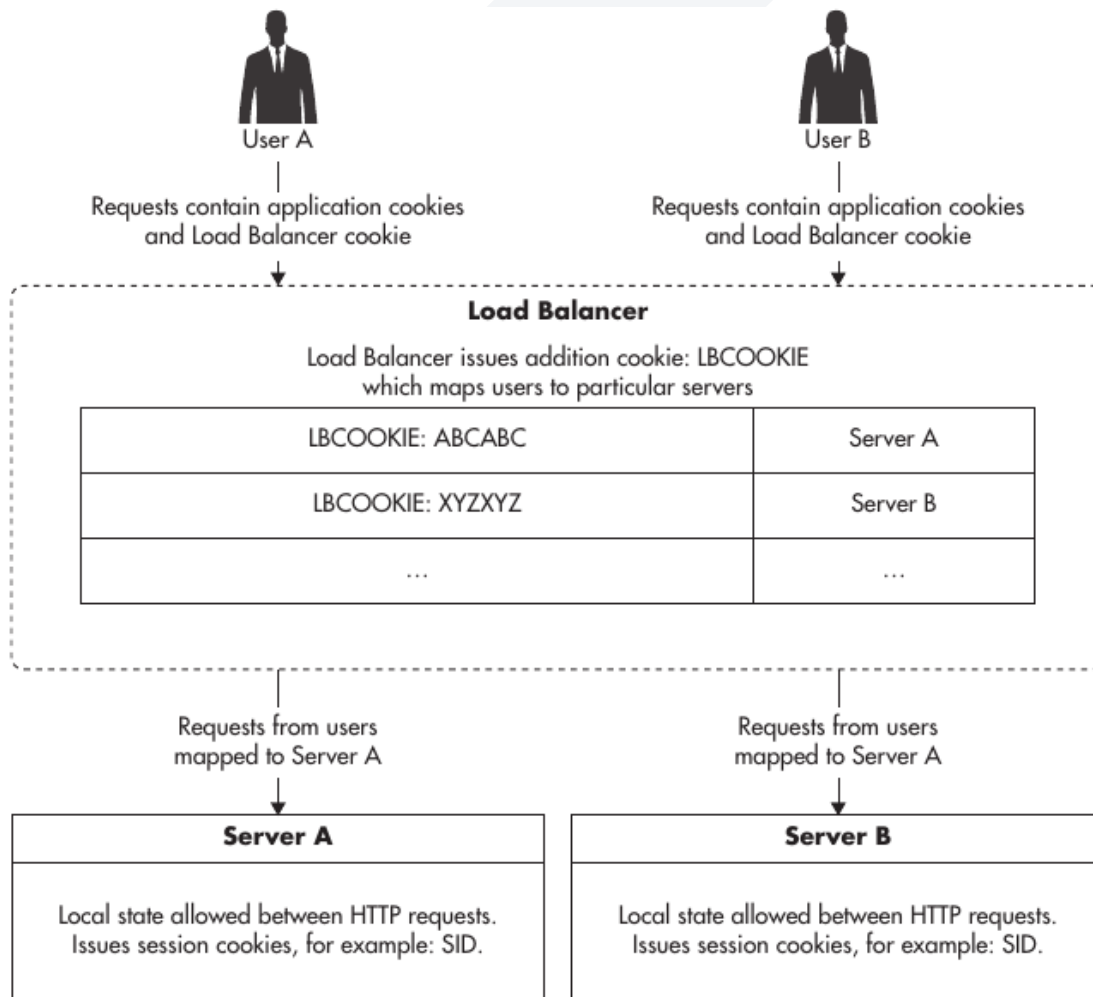
Session State



Load Balancer

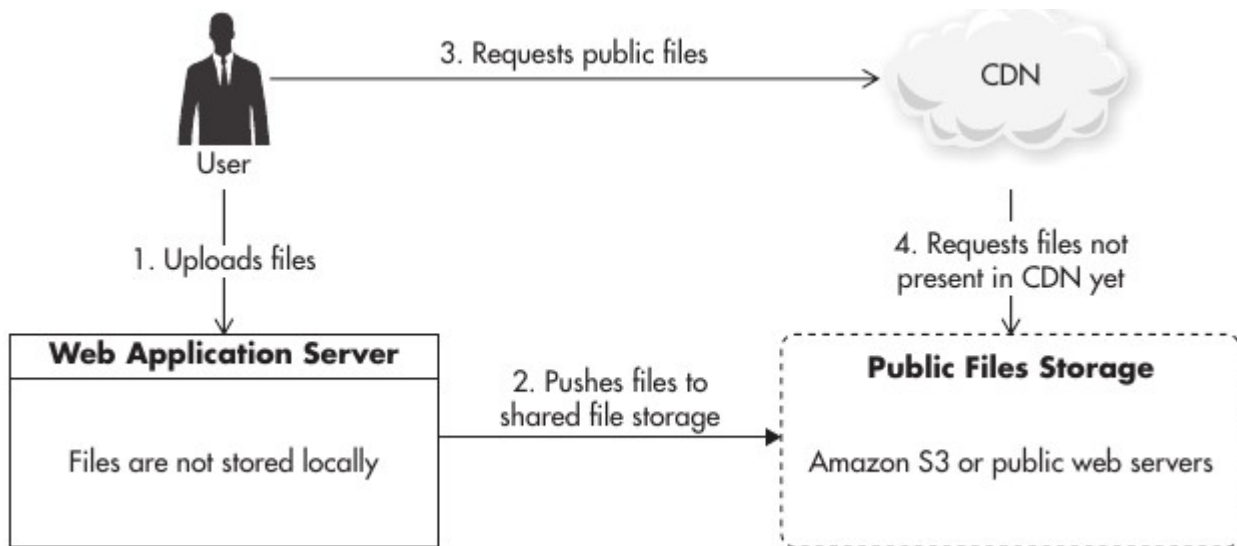
- Consiste em basicamente dar ao *load balancer* a responsabilidade de lidar com os estados de sessão.
 - Sempre que um novo cliente envia uma requisição, o *load balancer* atribui o cliente a um servidor específico e insere um novo *cookie* de *load balancer* na resposta.
 - Nas requisições seguintes o *load balancer* acessa os cabeçalhos da requisição para garantir que requisições com *cookies* da mesma sessão sempre sejam encaminhadas para o servidor específico. Essa prática é chamada de *sticky session*.
- Apesar de ser uma solução ao problema de gerenciamento de estado, essa solução faz com que a comunicação deixe de ser *stateless*.

Load Balancer



Arquivos

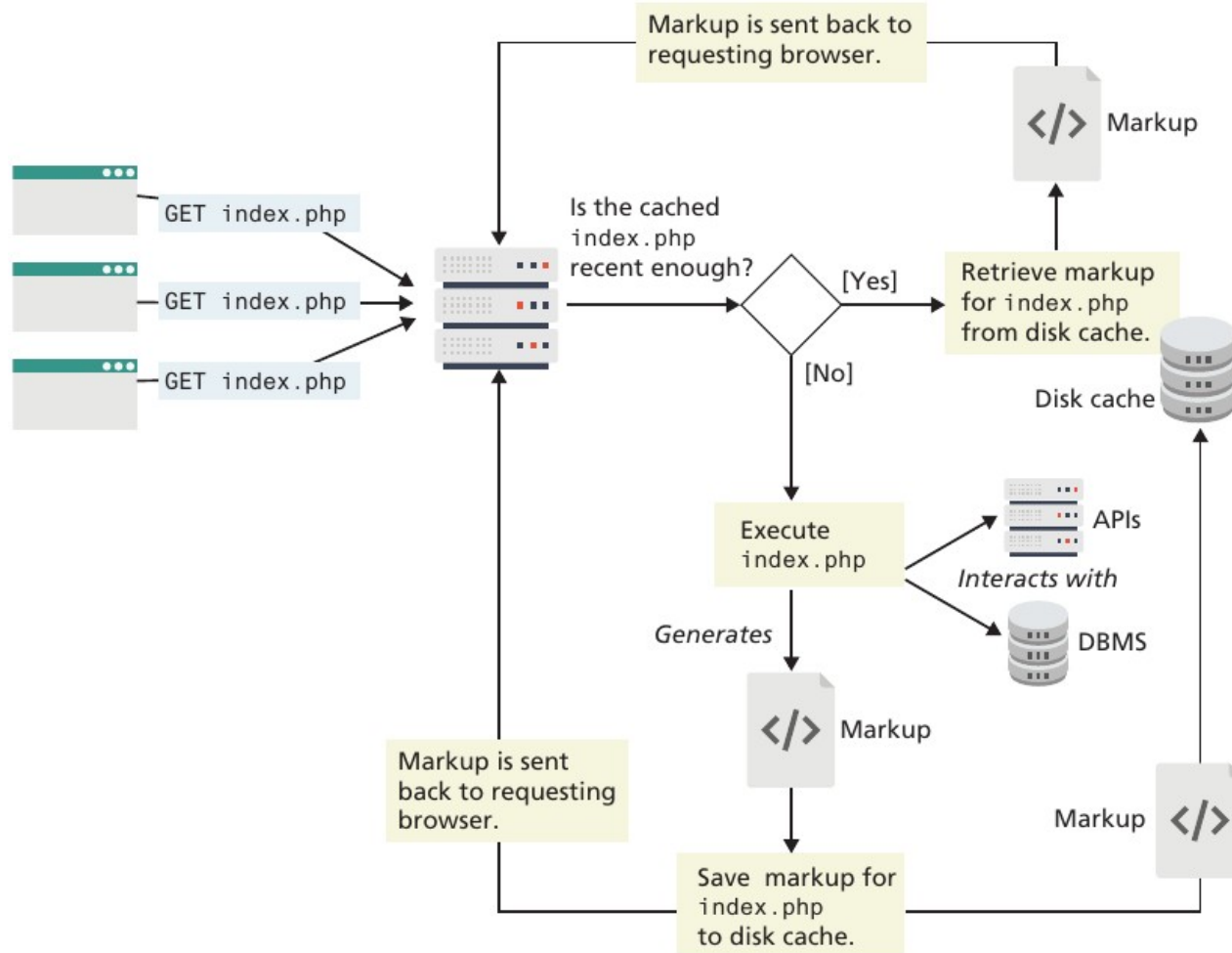
- Dois tipos de arquivo podem ser usados
 - Conteúdo gerado pelo usuário e enviado para os servidores.
 - Arquivos gerados pelo sistema que precisam ser baixados pelo usuário.



Caching

- *Cliente-side*
 - O navegador pode armazenar localmente imagens e outros arquivos, evitando novas requisições ao servidor.
 - *Web Storage API: localStorage e sessionStorage.*
- *Server-side*
 - *Page output caching*: salva a saída renderizada de uma página (ou parte dela) e reusa essa saída em vez de processar novamente requisições à ela.
 - *Data caching*: permite ao desenvolvedor cachear dados programaticamente.

Caching

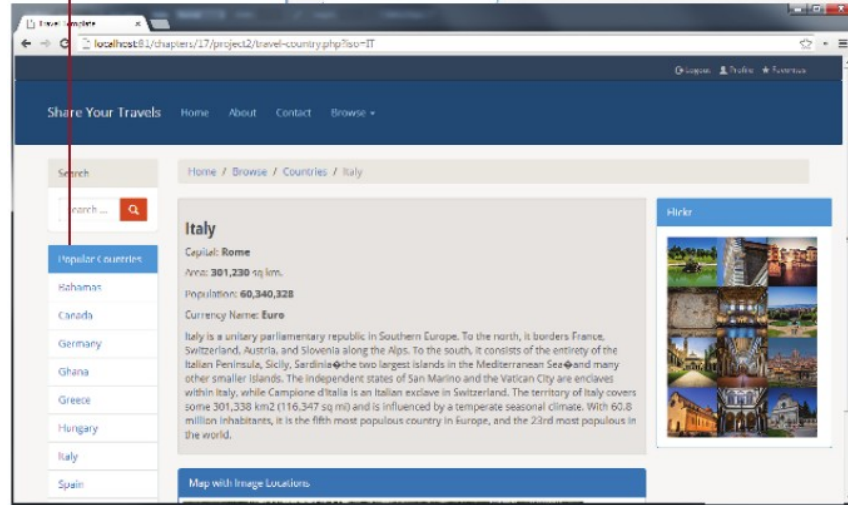
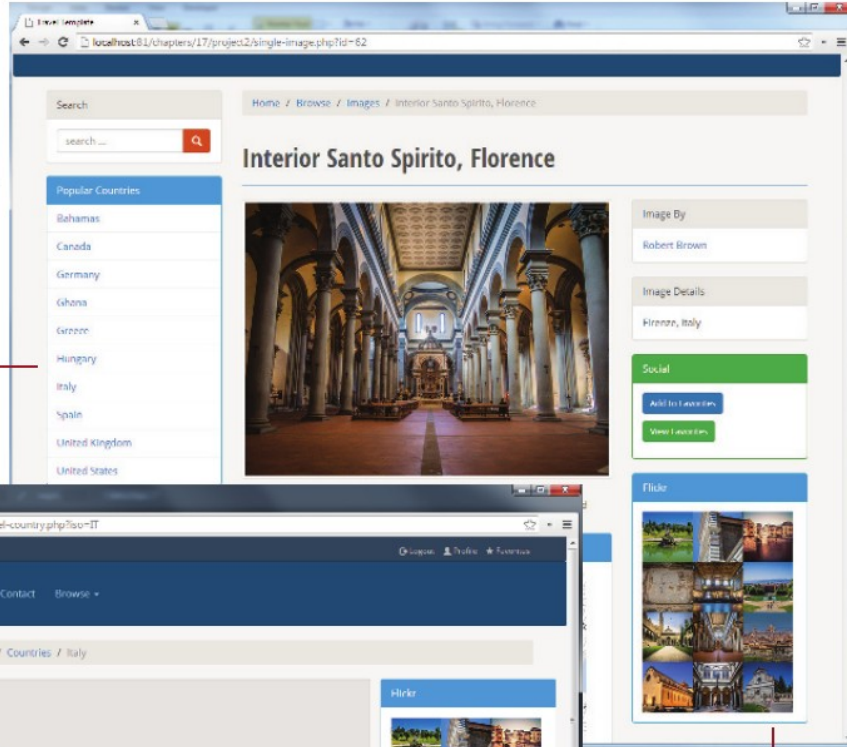


Caching

Imagine, every page in a site needs to run a SQL query to retrieve this list of countries from a table. That's a lot of queries executing again and again and again for a data set that rarely changes.



Each query: 50 ms



Each fetch: 450 ms



Imagine, multiple pages need to fetch data from an API that rarely changes. Again, this is a lot of additional fetches.

Caching

Data stored in server-side cache can be refreshed periodically to ensure it's accurate.

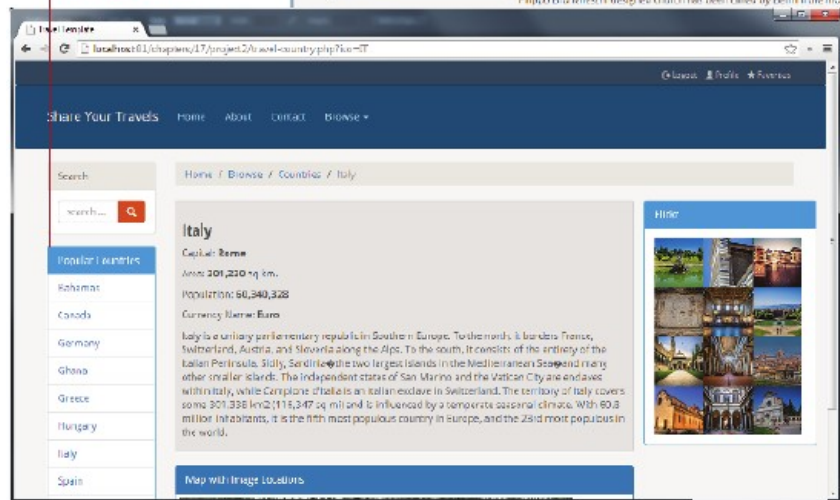
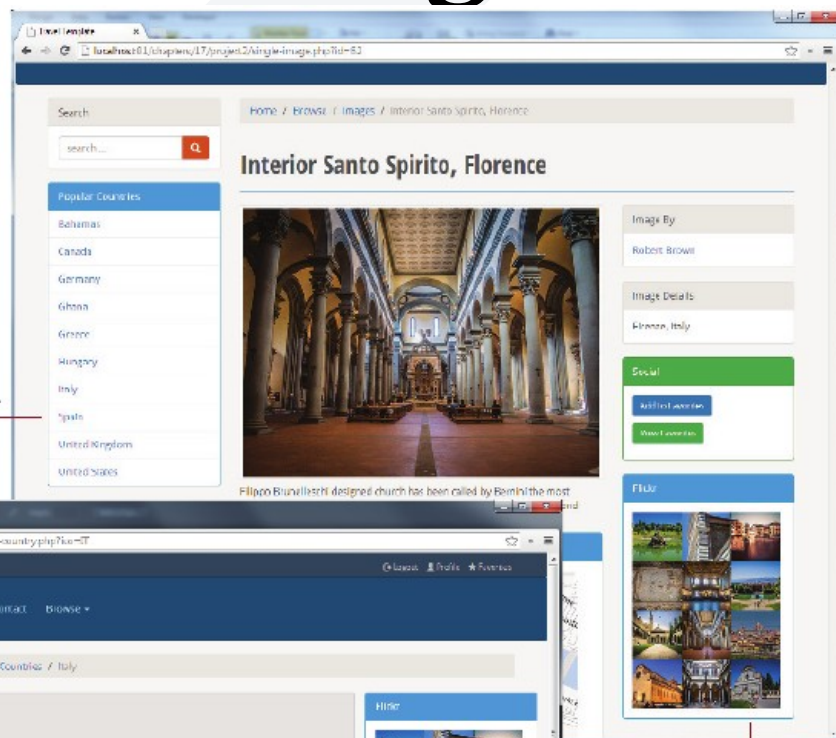


Query once: 50 ms



Pages can retrieve data from server cache very quickly.

Each server cache request: 5 ms



Each localStorage request: 5 ms



Fetch once: 450 ms

